

Rendu photoréaliste de maquettes architecturales pour l'étude de la lumière naturelle

**Stage de 3^{ème} année de
CMI Informatique Images, Réalité Virtuelle,
Interactions et Jeux**

Du 1er juin au 31 juillet 2026

Nicolas LAMY – Promotion 2023/24

Tuteur de stage : Paul VIVILLE,
Ingénieur de recherche, équipe IGG (ICube)

Responsable pédagogique :
Benoit NAEGEL

Remerciements

Je tiens tout d'abord à exprimer mes remerciements au laboratoire ICube et à l'équipe IGG pour m'avoir accueilli en tant que stagiaire pendant deux mois. Un remerciement particulier à mon maître de stage, Paul VIVILLE, pour sa confiance, son encadrement précieux et son expertise technique. Je le remercie de m'avoir guidé avec pédagogie dans l'appréhension des nouveaux outils et de m'avoir orienté vers les bonnes solutions d'optimisation. Ses conseils et son soutien ont été déterminants pour mener à bien mon travail. Je lui suis également très reconnaissant d'avoir accepté ma demande de stage et de m'avoir proposé un sujet aussi stimulant, en parfaite adéquation avec mes attentes et mes compétences.

Je souhaite également remercier Joris RAVAGLIA pour son accueil lors de mes premiers jours de stage et pour le temps qu'il m'a consacré lorsque Paul n'était pas présent. Ses retours sur mon projet m'ont permis de prendre du recul et d'améliorer certains aspects de mon travail.

Un grand merci à Quentin KLEIN, mon camarade de stage également encadré par Paul, pour notre excellente collaboration et son entraide au quotidien. Nos échanges techniques, notamment sur l'écriture de scripts C#, ont été très enrichissants et m'ont permis d'avancer plus efficacement. Je remercie également mes autres camarades stagiaires : Dylan, William, Elouan, Garance et Ilyes. Leur compagnie et leur expertise ont rendu cette période de stage à la fois productive et agréable. Au-delà des aspects techniques, les discussions informelles et les pauses partagées ont contribué à créer une ambiance de travail conviviale.

Je suis également très reconnaissant envers les autres membres de l'équipe IGG pour leur accueil chaleureux et leur soutien. Leur sympathie et leur disponibilité ont grandement contribué à la qualité de mon expérience de stage.

Je tiens également à exprimer ma gratitude envers Claire Karsenty, architecte et enseignante-chercheuse à l'École Nationale Supérieure d'Architecture de Strasbourg (ENSAS). Je la remercie chaleureusement de m'avoir accueilli au sein de son laboratoire pour discuter de mon projet, et de m'avoir fait découvrir leur simulateur physique de lumière naturelle sur maquette. Ces échanges concrets et très inspirants m'ont permis de mieux cerner les enjeux du projet et de comprendre les attentes réelles des architectes.

Mes sincères remerciements vont aussi à Benoit NAEGEL, responsable pédagogique de mon cursus CMI IIRVIJ. Son aide précieuse dans la recherche de ce stage, ainsi que son soutien continu, ont été essentiels pour que je puisse bénéficier de cette opportunité.

Enfin, je tiens également à exprimer ma gratitude envers tous mes enseignants et professeurs qui ont contribué à ma formation et m'ont soutenu dans mes démarches pour obtenir ce stage. Leur enseignement et leurs conseils m'ont été d'une grande aide pour réussir cette expérience professionnelle.

Table des matières

I - Introduction.....	4
II - Présentation du laboratoire.....	5
1 - Le laboratoire ICube.....	5
2 - L'équipe IGG	5
2.1 - Description Générale	5
2.2 - Thèmes de Recherche.....	5
III - Objectif du stage.....	6
1 - Objectif général	6
2 - Brève description des différentes étapes.....	6
2.1 – Prise en main et implémentation du modèle photoréaliste d'Unity.....	6
2.2 – Importation et traitement des maquettes 3D.....	6
2.3 – Tests et optimisation des performances	6
2.4 – Modélisation de la lumière naturelle via des données météorologiques	6
2.5 – Calcul précis sur la lumière sur rendu fixe	7
2.6 – Création d'un catalogue de lumières artificielles.....	7
2.7 – Ajout d'une interface utilisateur pour la gestion de la lumière en jeu.....	7
2.8 – Adaptation pour la réalité virtuelle dans un CAVE.....	7
IV – Méthodologie et Réalisation technique.....	8
1 - Mise en place du pipeline de rendu et intégration des maquettes	8
2 – Optimisation du rendu et des performances.....	9
2.1 – Impact de la géométrie et de l'albédo.....	9
2.2 – L'algorithme de la roulette russe et l'impact des paramètres.....	10
2.3 – Le compromis final et la transition fluide.....	10
3 – Simulation lumineuse dynamique et interactive	11
3.1 – Dynamisation de la lumière naturelle et données météorologiques.....	11
3.2 – Analyse quantitative et mesure de l'éclairage en temps réel.....	12
3.3 - Intégration d'un catalogue interactif de lumières artificielles	13
4 – Outils d'analyse et d'immersion	14
4.1 - Interface utilisateur et édition en temps réel.....	14
4.2 - Le portage stéréoscopique pour le CAVE.....	15
V - Résultats	16
VI - Analyse et discussion	17
1 – Adéquation avec le besoin métier	17
2 – Défis techniques et solutions apportées.....	17
3 – Bilan personnel et compétences acquises	18
VII - Conclusion.....	19
VIII – Références bibliographiques.....	20

I - Introduction

Ce rapport présente le déroulement de mon stage de deux mois au sein du laboratoire ICube, spécialisé dans les sciences de l'ingénieur, l'informatique et l'imagerie. L'objectif principal de ce stage était de développer, sur le moteur Unity, un prototype de visualiseur immersif permettant d'évaluer et de simuler l'impact des lumières naturelles et artificielles dans des projets architecturaux.

Travailler au sein de cette équipe pluridisciplinaire, tout en étant en étroite collaboration avec des architectes et enseignants-chercheurs de L'École nationale supérieure d'architecture de Strasbourg (ENSAS), et plus particulièrement Claire Karsenty, m'a offert une opportunité unique de lier développement informatique et enjeux écologiques et muséaux. Mon rôle a consisté à concevoir un outil capable d'importer des maquettes numériques et d'en générer une version photoréaliste explorable à l'échelle 1:1. J'ai dû intégrer une double dimension analytique : qualitative, pour l'expérience réaliste et perceptive des usagers, et quantitative, impliquant la modélisation et mesure de l'illumination naturelle et l'intégration de catalogues de luminaires artificiels.

Ce stage a été une occasion précieuse pour approfondir mes compétences techniques, tout particulièrement le développement en C# sous Unity, l'exploitation des outils de rendu photoréaliste (comme le *Ray-Traced Global Illumination* et le *Path Tracing*) et la création d'expériences en réalité virtuelle. J'ai également été confronté aux défis d'optimisation des performances graphiques en temps réel. Travailler dans ce contexte de recherche m'a permis de découvrir un environnement professionnel stimulant et de contribuer au développement d'un outil ayant un réel impact sur la conception de bâtiments plus confortables et moins énergivores.

Ainsi, ce rapport se propose de détailler les différentes étapes de conception de ce prototype, les défis techniques rencontrés, notamment les compromis entre qualité de rendu lumineux et fluidité en réalité virtuelle et les compétences acquises, tout en soulignant l'importance de cette expérience pour mon développement personnel et professionnel.

II - Présentation du laboratoire

1 - Le laboratoire ICube

Le laboratoire ICube ¹ est une unité mixte de recherche située à Strasbourg, dont le principal site se situe sur le campus d'Illkirch-Graffenstaden au Pôle API. Ce laboratoire est affilié à l'Université de Strasbourg, au CNRS (Centre National de la Recherche Scientifique), à l'ENGEEES (École Nationale du Génie de l'Eau et de l'Environnement de Strasbourg) et à l'INSA (Institut National des Sciences Appliquées) de Strasbourg. C'est une structure d'envergure qui rassemble aujourd'hui plus de 700 membres (chercheurs, enseignants-chercheurs, ingénieurs, doctorants et personnels administratifs). Le laboratoire ICube a été créé en 2013 à la suite de la fusion de plusieurs unités de recherche et est structuré autour de quatre grands pôles de compétence :

- Imagerie
- Informatique
- Robotique
- Télécommunications

ICube est reconnu pour sa recherche interdisciplinaire qui couvre un large éventail de domaines allant des technologies de l'information et de la communication à la santé, en passant par l'environnement et l'énergie. Le laboratoire mène des projets de recherche fondamentaux et appliqués, souvent en collaboration avec des partenaires industriels et académiques nationaux et internationaux.

2 - L'équipe IGG

2.1 - Description Générale

L'équipe IGG ² (Informatique Géométrique et Graphique) fait partie intégrante du laboratoire ICube et compte à ce jour environ une quarantaine de membres actifs, répartis entre chercheurs, enseignants-chercheurs, ingénieurs et doctorants. Ses recherches se concentrent sur la modélisation géométrique, la visualisation scientifique, et les interactions homme-machine, avec une forte composante algorithmique et computationnelle.

Les travaux de l'équipe visent à développer des méthodes innovantes pour représenter, analyser et interagir avec des données géométriques complexes, que ce soit en 2D ou en 3D. Ces avancées trouvent des applications dans des domaines variés, allant de la médecine à l'ingénierie, en passant par la réalité virtuelle et la visualisation de données massives.

2.2 - Thèmes de Recherche

- *Preuves et modélisation en géométrie* : Développement de modèles mathématiques et de méthodes formelles pour la spécification, la construction et la validation rigoureuse des propriétés géométriques et topologiques des objets numériques.
- *Géométrie 3D et animation* : Conception d'algorithmes pour la création, la manipulation, l'optimisation et l'animation de structures géométriques et de formes 3D complexes, adaptées à divers environnements virtuels.
- *Textures, rendu et visualisation* : Utilisation de processus stochastiques et de méthodes de génération procédurale (synthèse de textures, gestion de la lumière) pour le rendu visuel performant (souvent en temps réel) et la visualisation de grandes masses de données.
- *Interaction homme-machine et réalité virtuelle* : Développement d'interfaces immersives et de simulations en réalité virtuelle ou augmentée, notamment appliqués pour des usages médicaux (comme les thérapies).

En conclusion, l'équipe IGG du laboratoire ICube joue un rôle essentiel dans l'avancement de l'informatique graphique, de la simulation 3D et de la réalité virtuelle. Grâce à une combinaison d'approches méthodologiques innovantes et une forte collaboration avec des secteurs variés (médecine, industrie), l'équipe contribue de manière significative aux avancées technologiques et scientifiques permettant de lier l'humain aux environnements virtuels.

III - Objectif du stage

1 - Objectif général

L'objectif principal de ce stage était de concevoir et développer un prototype de visualiseur 3D immersif sous le moteur Unity³. Cet outil est destiné aux architectes et concepteurs d'espaces (notamment muséaux) pour leur permettre d'importer une maquette numérique et d'en générer une version photoréaliste explorable à l'échelle 1:1. Il répond plus spécifiquement à un besoin identifié par Claire Karsenty pour l'étude des espaces muséaux, où la perception des œuvres est étroitement liée à la maîtrise de leur éclairage (naturel comme artificiel).

La finalité de ce visualiseur est d'évaluer l'impact et la répartition de la lumière, qu'elle soit naturelle ou artificielle, à la fois d'un point de vue qualitatif (confort visuel, perception de l'espace) et quantitatif (mesure de l'intensité, ratio naturel/artificiel). Pour y parvenir, l'application devait s'appuyer sur des données physiques réelles et utiliser des techniques de rendu de pointe afin d'être la plus fidèle possible à la réalité physique, tout en maintenant des performances compatibles avec une exploration en temps réel et, à terme, en réalité virtuelle.

2 - Brève description des différentes étapes

2.1 - Prise en main et implémentation du modèle photoréaliste d'Unity

La première étape a consisté à appréhender le *High Definition Render Pipeline* (HDRP) d'Unity. Il m'a fallu étudier et comparer différentes méthodes d'illumination globale, en particulier le Path Tracing (qui offre un rendu physiquement réaliste mais très coûteux en ressources) et le *Ray-Traced Global Illumination* (RTGI), afin d'établir un premier référentiel de performances (FPS, temps de convergence) sur des formes simples.

2.2 - Importation et traitement des maquettes 3D

Il s'agissait de récupérer des maquettes brutes fournies par des architectes et de les préparer pour Unity. Cette étape impliquait un passage par Blender pour convertir les modèles en .FBX (plus facilement manipulable sur Unity), séparer les différents maillages (murs, tissus, verre) et configurer proprement le texturage et l'UV mapping (notamment par projection triplanaire). Toutes ces configurations serviront à créer une scène idéale et représentative pour réaliser mon projet.

2.3 - Tests et optimisation des performances

L'intégration du *Path Tracing* dans un espace architectural soulevant des défis matériels majeurs, une phase d'analyse de performances a été indispensable. L'objectif était de mesurer systématiquement l'impact des différents paramètres physiques et des matériaux sur le temps de rendu. Ces tests visaient à comprendre le comportement interne du processeur graphique afin d'établir le meilleur compromis technique. Cela a également motivé la mise en place de scripts de transition fluide et l'intégration d'outils d'optimisation pour garantir une expérience utilisateur sans saccades.

2.4 - Modélisation de la lumière naturelle via des données météorologiques

L'outil devait refléter la réalité géographique du bâtiment. J'ai donc développé un système communiquant avec l'API européenne PVGIS (*Photovoltaic Geographical Information System*) pour récupérer les données d'irradiation solaire historiques en fonction des coordonnées géographiques, de la date et de l'heure, afin de piloter dynamiquement l'intensité du soleil et du ciel dans la scène.

2.5 – Calcul précis sur la lumière sur rendu fixe

Pour répondre au besoin quantitatif du projet, une étape a été dédiée à la récupération des véritables valeurs de luminance sur la maquette. Les cartes de lumière (*lightmaps*) classiques étant inadaptées à un éclairage dynamique, il a fallu mettre en place un système capable de lire directement les valeurs des pixels sur la caméra.

2.6 – Création d'un catalogue de lumières artificielles

Afin d'évaluer la nécessité ou non d'un apport artificiel, j'ai intégré à la scène des lumières artificielles IES (*Illuminating Engineering Society*) basées sur des profils photométriques normalisés. Cela a permis de créer un contraste physique réel entre les apports solaires et les luminaires d'intérieur.

2.7 – Ajout d'une interface utilisateur pour la gestion de la lumière en jeu

Pour rendre le prototype utilisable, j'ai implémenté une interface utilisateur (UI) intuitive permettant de modifier facilement les paramètres de la simulation : choix de la date et de la position, modification de l'heure locale, gestion du placement des lumières artificielles, et retours visuels.

2.8 – Adaptation pour la réalité virtuelle dans un CAVE

L'objectif à terme était de déployer ce prototype dans le CAVE (*Cave Automatic Virtual Environment*) d'ICube. Cette étape impliquait l'installation de paquets spécifiques pour la stéréoscopie, la configuration de caméras multiples, et l'adaptation du système de déplacement pour pallier les problèmes de bruit visuel induits par le micro-suivi des mouvements de tête.

IV – Méthodologie et Réalisation technique

1 - Mise en place du pipeline de rendu et intégration des maquettes

La première étape technique du projet a consisté à mettre en place l'environnement de travail dans Unity et à y intégrer les données architecturales.

Pour atteindre l'objectif de photoréalisme, j'ai configuré le projet sous le *High Definition Render Pipeline* ⁴ (HDRP) d'Unity. Contrairement au pipeline standard ou à l'URP (*Universal Render Pipeline*) conçus pour être légers et tourner sur des supports mobiles, le HDRP est pensé pour les architectures PC avec des hautes performances. Son fonctionnement repose sur le PBR (*Physically Based Rendering*), un modèle mathématique simulant la véritable interaction entre la lumière et les matériaux (rugosité, réflexion métallique, réfraction). De plus, ce pipeline débloque l'accès à des technologies avancées de calcul de la lumière comme le *Path Tracing*, qui nous sera très utile ici pour simuler le comportement naturel des rayons lumineux.



Figure 1 : Comparaison du rendu des modèles HDRP et URP sur le jeu Kona II : Brume ([source](#))

Une fois l'environnement configuré, il a fallu traiter les modèles 3D. Initialement, j'ai reçu de la part des architectes partenaires une maquette brute d'une pièce d'un musée au format STL, exportée depuis le logiciel SketchUp. Pour mes premiers tests d'intégration, j'ai pu faciliter l'import de ce fichier dans mon projet Unity grâce à un dépôt public sur GitHub ⁵ (le moteur de jeu ne supportant pas nativement ce format de fichier). C'est avec cette exemple de maquette que j'effectuerais la plupart de mes travaux sur ce projet.

Cependant, si ce format est très utilisé en conception architecturale, il n'est pas optimisé pour les moteurs de rendu en temps réel : il ne conserve pas les informations de texturage et fusionne souvent la géométrie. En effet, si nous devions n'exploiter que la lumière naturelle sur la structure architecturale depuis un point de vue extérieur, ce format aurait été suffisant. Or, l'intérêt principal du projet était de pouvoir importer des maquettes avec différentes textures sur les surfaces (murs, œuvres, sols) mais également des lumières artificielles, visibles en intérieur, dans le but d'avoir un rendu le plus proche de la réalité.

Ainsi, afin de rendre ce modèle pleinement exploitable pour mon projet dans Unity, j'ai mis en place un processus de préparation via le logiciel Blender.

J'ai, tout d'abord, séparé manuellement les différents éléments de la géométrie de la maquette (murs, sol, plafond) afin de pouvoir leur appliquer plus facilement des matériaux aux propriétés physiques distinctes dans le moteur de jeu.

Les modèles architecturaux présentent souvent des géométries complexes (angles, renforcements). Pour éviter que les textures ne soient étirées ou déformées lors de leur application, j'ai effectué une projection triplanaire.

Au lieu de déplier manuellement le maillage de manière complexe, cette méthode projette la texture simultanément depuis les trois axes de l'espace (X, Y et Z), assurant un rendu visuel homogène et continu sur l'ensemble des surfaces.

J'ai, enfin, exporté cette maquette au format FBX, qui est le standard pour Unity, et, avec lequel, j'avais déjà travaillé de nombreuses fois sur des projets précédents.

Enfin, une fois la maquette importée dans Unity, la dernière étape de cette phase a été de configurer les matériaux. Il a fallu convertir les textures standards pour qu'elles soient compatibles avec le moteur HDRP. J'ai notamment porté une attention particulière aux surfaces vitrées (certains murs de la maquette) : j'ai ajusté leurs paramètres de transparence et de réfraction, et j'ai désactivé la projection d'ombres sur ces éléments pour permettre aux rayons du soleil de pénétrer correctement à l'intérieur de la maquette, préparant ainsi le terrain pour les calculs d'illumination.

2 – Optimisation du rendu et des performances

L'utilisation du *Path Tracing* permet un rendu physiquement exact, mais elle est extrêmement coûteuse en ressources de calcul. Ce coût s'explique par le principe même de l'algorithme : pour chaque pixel de l'écran, le moteur lance de multiples rayons lumineux qui vont rebondir sur les surfaces de la scène, un certain nombre de fois avant d'atteindre une source de lumière.

Afin de trouver le meilleur compromis entre la fluidité de l'application et la qualité visuelle, j'ai mis en place un protocole de test rigoureux (consigné dans un tableau, figure 2) pour mesurer l'impact de chaque paramètre sur le processeur graphique.

Test de perf du Path Tracing						
Max Depth (Rebonds)	Max Samples (Echantillons)	GPU (ms)	FPS en mouvement	Temps de convergence statique		
2	256		#DIV/0!	0,00		
4	256		#DIV/0!	0,00	Résolution	1920x1080
6	256		#DIV/0!	0,00	Denoiser NVIDIA	Activé
12	256		#DIV/0!	0,00	Nb triangles	
6	64		#DIV/0!	0,00	Nb points	
6	128		#DIV/0!	0,00	Batches	
6	512		#DIV/0!	0,00	Setpass calls	
6	1024		#DIV/0!	0,00		
12	1024		#DIV/0!	0,00	cas extrême	

Figure 2 : Tableau vierge de tests de performances du Path Tracing

Le principe était le suivant :

- je modifie les valeurs du nombre maximum d'échantillons (rayons) et de rebonds du Path Tracing
- je me déplace dans la scène tout en mesurant les performances de la carte graphique (en ms)
- j'importe les résultats dans le tableau, qui calcule automatiquement les FPS en mouvement et le temps de convergence statique, c'est-à-dire le temps que met l'image à se débruiter une fois que l'on s'arrête de bouger (image fixe)

Ces tests serviront à trouver le bon compromis entre fluidité d'affichage et réalisme du rendu lorsque nous devrons nous déplacer dans la maquette.

2.1 – Impact de la géométrie et de l'albédo

Lors de mes premiers tests sur des formes simples en extérieur (figure 3), le prototype tournait à environ 180 images par seconde (FPS). Cependant, une fois la maquette architecturale importée (figure 4), les performances ont chuté autour de 45 FPS. Cette baisse est logique : dans un espace clos, la lumière reste piégée et rebondit de nombreuses fois sur les murs, ce qui démultiplie les calculs.

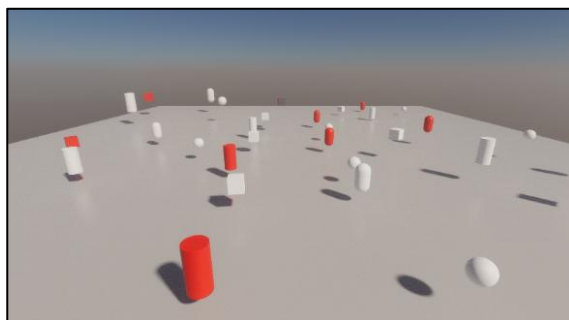


Figure 3 : Scène de tests avec formes simples

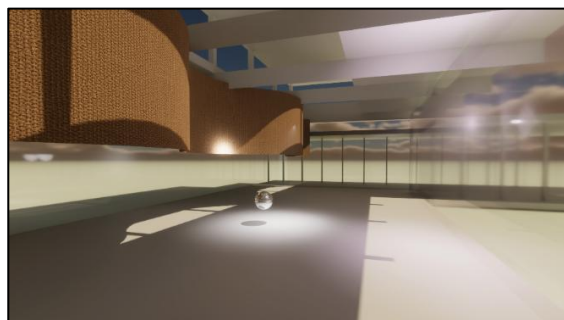


Figure 4 : Scène de tests avec maquette et textures

J'ai également pu constater l'impact direct de la couleur des matériaux (albédo) sur les performances. Par exemple, avec des murs sombres ou avec des couleurs chaudes, les rayons perdaient rapidement leur énergie par absorption thermique simulée, ce qui allégeait le calcul. À l'inverse, en appliquant des textures claires (presque blanches) plus fidèles à l'architecture muséale, les rayons conservaient leur énergie et continuaient de rebondir, saturant le GPU et faisant chuter les performances.

2.2 - L'algorithme de la roulette russe et l'impact des paramètres

Pour éviter que les calculs ne deviennent infinis avec des textures claires, Unity utilise une technique d'optimisation interne appelée algorithme de la roulette russe. Il évalue aléatoirement s'il doit "tuer" un rayon lumineux qui a perdu trop d'énergie au fil de ses rebonds, tout en augmentant artificiellement l'intensité des rayons survivants pour garantir un résultat final visuellement non biaisé. C'est grâce à cette technique qu'une limite de 12 rebonds maximum s'est parfois révélée autant, voire plus rapide, à calculer qu'une limite de 6 rebonds dans mes tests (comme l'illustrent les graphiques de la figure 5).

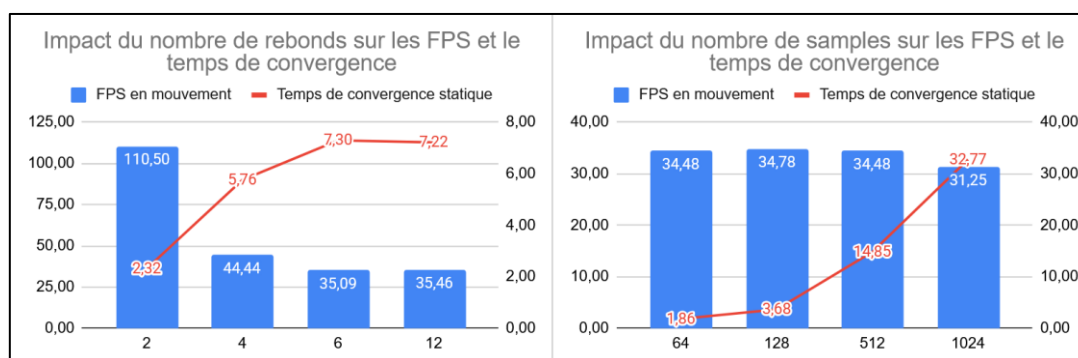


Figure 5 : Graphiques montrant l'impact des paramètres du Path Tracing sur le temps de convergence

Cependant, il n'était pas possible de réduire le nombre de rebonds de manière trop agressive. J'ai remarqué qu'avec un paramètre fixé à très peu de rebonds, la lumière extérieure ne parvenait plus à éclairer la pièce. En effet, le rayon "mourait" à l'intérieur même de l'épaisseur de la vitre architecturale avant de pouvoir en ressortir.

2.3 - Le compromis final et la transition fluide

À l'issue de ces tests, j'ai identifié un compromis idéal : un nombre fixé à 256 échantillons et une limite de 6 rebonds. Cette configuration permettait d'obtenir une image fixe parfaite en 7 à 14 secondes d'attente, tout en maintenant des performances acceptables lors des déplacements.

Néanmoins, le Path Tracing génère un "bruit" visuel (des pixels granuleux) très désagréable lors des mouvements, le temps que l'image converge. Pour garantir une expérience utilisateur fluide, j'ai développé un script de transition dynamique, couplé à la technologie de mise à l'échelle DLSS (*Deep Learning Super Sampling*) de NVIDIA. Cette méthode utilise l'intelligence artificielle pour calculer l'image à une définition inférieure puis la reconstruire à la résolution native, ce qui allège la charge sur le processeur graphique et augmente considérablement la fluidité.

Lorsque l'utilisateur se déplace, le moteur bascule sur un éclairage global moins coûteux (*Ray-Traced Global Illumination* - RTGI). Lorsqu'il s'arrête, le Path Tracing s'active. Pour masquer le léger bruit de chargement durant cette transition, j'ai programmé un effet visuel simulant l'ajustement de la focale et de l'exposition d'un appareil photo (fade in / fade out avec flou de mise au point), rendant le passage entre les deux méthodes de calcul presque imperceptible.

3 – Simulation lumineuse dynamique et interactive

L'objectif central du projet n'était pas seulement d'obtenir un beau rendu visuel, mais de fournir un vrai outil d'analyse quantitative et qualitative. Pour cela, la lumière ne pouvait pas être "statique" ou peinte artificiellement : elle devait réagir dynamiquement aux conditions réelles et pouvoir être mesurée avec précision.

3.1 – Dynamisation de la lumière naturelle et données météorologiques

Pour que la maquette reflète son futur environnement réel, l'éclairage naturel devait correspondre aux véritables coordonnées géographiques du bâtiment représenté en maquette, à la date et à l'heure souhaitées.

J'ai d'abord implémenté un système de positionnement mathématique du soleil via un script C# (voir figure 6). À l'aide de formules astronomiques ⁶, le script calcule en temps réel l'élévation et l'azimut du soleil. Il prend notamment en compte la déclinaison de la Terre selon la saison et l'équation du temps ⁷ (correction entre le temps solaire vrai et le temps moyen).

Cependant, la position du soleil ne suffit pas : il faut aussi simuler l'intensité lumineuse réelle (nuages, saisons, etc.). J'ai donc développé un gestionnaire qui communique directement avec l'API européenne PVGIS ⁸ (*Photovoltaic Geographical Information System*). Ce script télécharge un fichier JSON contenant, entre autres, les données d'irradiation solaire historiques d'un lieu donné (voir figure 7). Ces données sont fournies par l'API via son générateur d'année météorologique type ⁹ (TMY, *Typical Meteorological Year*) en ayant entré les coordonnées GPS de l'endroit où nous voulons nous situer dans l'URL envoyé en Web Request par Unity à l'API.

Une année météorologique type est un ensemble de données météorologiques comprenant les valeurs enregistrées toutes les heures au cours d'une année pour un lieu géographique donné. Ces données sont sélectionnées à partir des données horaires disponibles pour l'ensemble de la période concernée, à savoir actuellement 2005-2020 dans PVGIS 5.2.

J'ai dû concevoir un parseur pour nettoyer et extraire deux valeurs fondamentales :

- La radiance directe : les rayons du soleil qui frappent directement le bâtiment. Je l'ai traduite en Lux dans Unity pour piloter l'intensité de la source de lumière directionnelle (soleil).
- La radiance diffuse : la lumière dispersée par l'atmosphère (les nuages, le ciel). Elle me permet de piloter l'exposition du volume du ciel entourant la maquette.

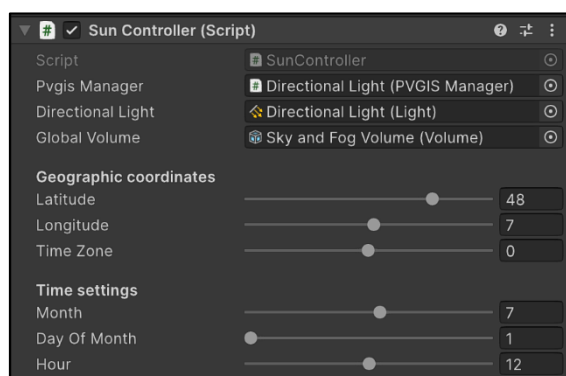


Figure 6 : Outil de gestion de la position du Soleil depuis l'inspecteur d'Unity

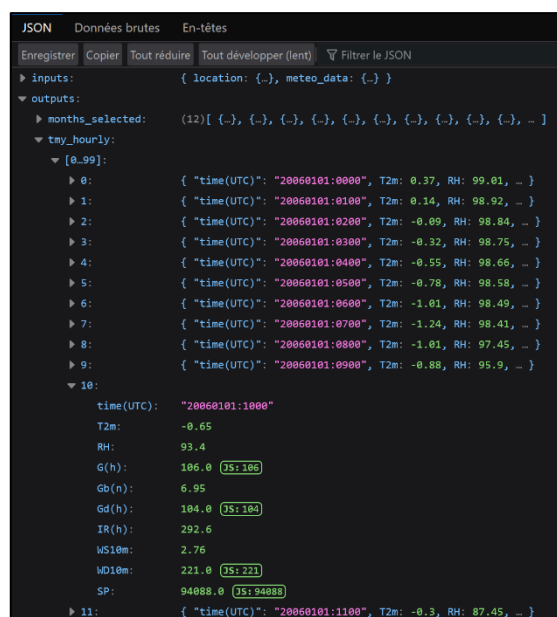


Figure 7 : Aperçu des données météorologiques récupérées depuis l'API PVGIS

Afin d'assurer des transitions fluides lorsque l'utilisateur fait défiler le temps, j'ai programmé une interpolation linéaire entre les données météorologiques d'une heure à l'autre. J'ai également dû gérer techniquement les cas d'ensoleillement extrême ou très faible. Par exemple, par temps très couvert, la radiance directe est quasi nulle : La majorité de la lumière provient du ciel. Pour éviter que le ciel ne devienne visuellement noir tout en conservant les calculs physiques, j'ai instauré un seuil de sécurité minimal géré par le moteur de rendu de manière proportionnelle au ratio entre la radiance directe réelle et le seuil minimal imposé.

3.2 – Analyse quantitative et mesure de l'éclairage en temps réel

Une fois la lumière correctement simulée, le projet exigeait de pouvoir mesurer son intensité sur les surfaces (sol, œuvres, murs).

Dans un premier temps, j'ai exploré l'utilisation des lightmaps d'Unity (des textures pré-calculées stockant la lumière, voir figure 8). Si cette méthode est performante, je me suis vite heurté à un problème majeur : Les lightmaps sont statiques. Si l'on modifie l'heure de la journée ou que l'on déplace une lampe, les valeurs calculées deviennent fausses. Il était impératif de conserver la souplesse du rendu en temps réel fourni par le Path Tracing.

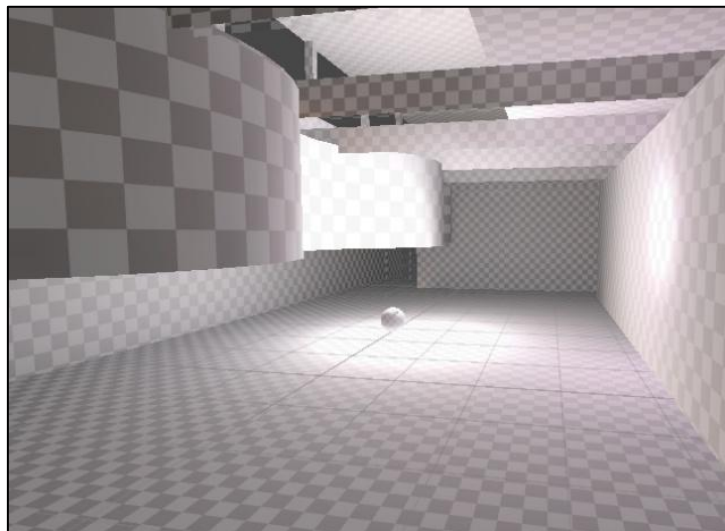


Figure 8 : Aperçu des lightmaps de la maquette

J'ai donc changé d'approche en développant une sonde de luminance virtuelle. L'idée est de lire directement les informations contenues dans les pixels de l'image rendue par la caméra. Pour que cela fonctionne, il était crucial de capturer l'image au format HDR (*High Dynamic Range*). Contrairement aux images classiques dont les couleurs sont plafonnées à une valeur de 1 (blanc pur), le HDR permet d'enregistrer les vraies valeurs d'énergie lumineuse, bien au-delà de 1.

Lorsqu'un utilisateur clique sur une surface de la scène, le script identifie le pixel correspondant et extrait ses valeurs de couleur brute (Rouge, Vert, Bleu). J'applique ensuite la formule photométrique standard de conversion colorimétrique ¹⁰ ($Y = 0.2126 R + 0.7152 G + 0.0722 B$) pour transformer ces couleurs en une valeur de luminance exacte en candela par mètre carré (cd/m^2).

Pour répondre aux besoins d'étude de l'architecte, il fallait aller plus loin et distinguer l'origine de cette lumière. J'ai donc conçu le système pour qu'il effectue un double relevé :

- La luminance totale (toutes les lumières allumées).
- La luminance naturelle (en isolant uniquement l'impact du soleil et du ciel).

Il en déduit mathématiquement la luminance artificielle par une simple soustraction (totale - naturelle).

À partir de ces valeurs, le script calcule en temps réel le ratio ($\text{luminance naturelle} / \text{luminance totale} * 100$).

Pour m'assurer que mes calculs algorithmiques étaient exacts et non faussés par des rebonds parasites, j'ai créé une scène de test "boîte noire" (voir figure 9) totalement hermétique. J'y ai placé deux sources lumineuses aux caractéristiques connues : l'une simulant un apport naturel (carré lumineux) et l'autre un apport artificiel (faisceau diffus).

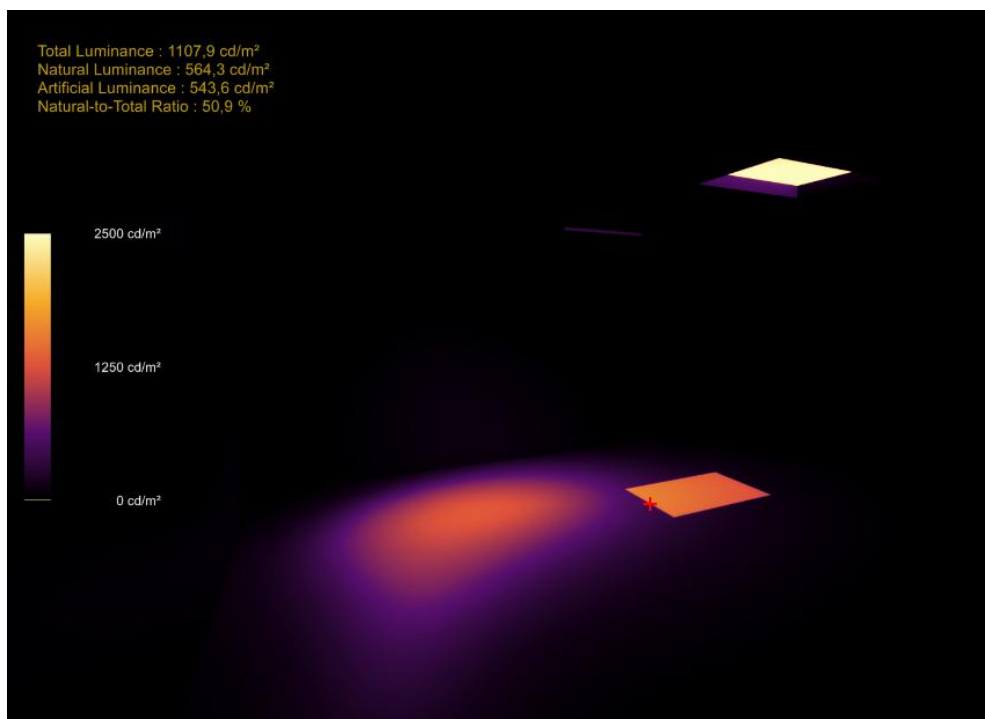


Figure 9 : Test des validations de calcul dans un environnement « boîte noire »

À l'aide d'un compute shader, j'ai programmé une carte de chaleur qui analyse toute l'image via la carte graphique. L'échelle va du noir/violet (0 cd/m²) au jaune clair (ici paramétré jusqu'à 2500 cd/m²). Comme le montre l'image de test, lorsque qu'on clique dans la zone où les deux lumières se croisent (là où se trouve le curseur rouge), l'outil décompose parfaitement les apports : 564,3 cd/m² de lumière naturelle et 543,6 cd/m² de lumière artificielle, donnant un total de 1107,9 cd/m² et un équilibre (ratio) de 50,9 %. Ce test en milieu isolé a permis de valider la précision de l'outil avant de l'appliquer à la maquette complète du musée.

3.3 - Intégration d'un catalogue interactif de lumières artificielles

Pour compléter l'étude du ratio entre lumière naturelle et artificielle, il fallait permettre à l'architecte de tester différents dispositifs d'éclairage intérieur.

J'ai conçu un système de placement dynamique permettant à l'utilisateur de sélectionner des lampes depuis une interface et de les positionner à la volée sur les surfaces de la maquette (plafonds, murs). Pour que ces lumières soient réalistes et non de simples sources lumineuses génériques, je me suis appuyé sur les profils photométriques standardisés du catalogue de données ouvertes IES ¹¹ (*Illuminating Engineering Society*).

Ces fichiers, fournis par les vrais fabricants d'éclairage, modélisent la forme complexe du faisceau lumineux et sa dispersion spatiale (voir figure 10). J'ai structuré ce catalogue sous forme de scriptable objects dans Unity, ce qui permet d'ajouter très facilement de nouveaux modèles de lampes sans toucher au code. Enfin, j'ai calibré l'intensité de ces lampes autour de valeurs réalistes afin de pouvoir observer correctement leur impact face à l'écrasante puissance de la lumière solaire calculée précédemment.

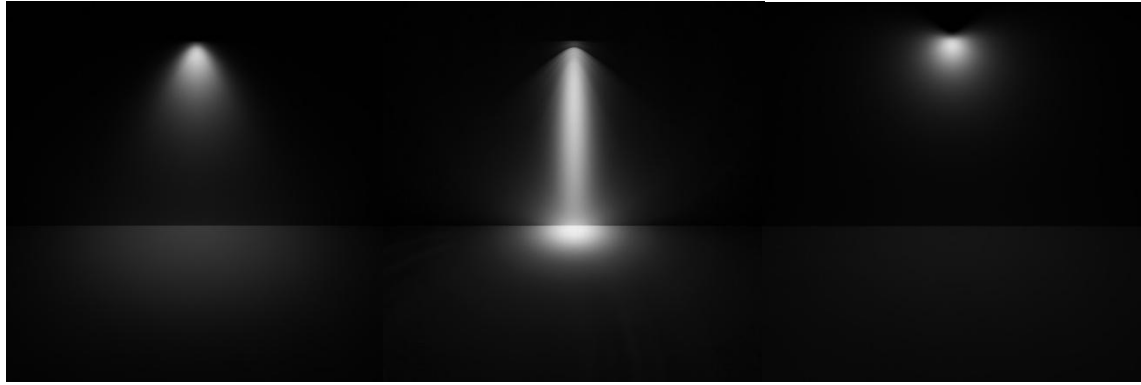


Figure 10 : Exemples de luminaires IES présents dans le catalogue

4 – Outils d'analyse et d'immersion

Pour que ce prototype devienne un véritable outil d'aide à la conception, il ne suffisait pas que les calculs soient exacts : il fallait que l'architecte puisse interagir avec la maquette intuitivement, mais aussi qu'il puisse s'y immerger à l'échelle 1:1.

4.1 - Interface utilisateur et édition en temps réel

Les professionnels (architectes, conservateurs) n'ont pas vocation à utiliser l'éditeur complexe d'Unity dans sa totalité. J'ai donc développé une interface utilisateur accessible directement dans l'application finale (voir figure 11). Cette interface se divise en deux volets principaux.

Tout d'abord, un premier onglet pour modifier les paramètres environnementaux a été créé. L'utilisateur peut modifier la date, l'heure, la latitude et la longitude via des curseurs. Le soleil (et l'irradiation associée) se met à jour instantanément après avoir appuyé sur un bouton pour charger la position (afin d'éviter de renvoyer un message à l'API PVGIS à chaque frame). C'est également ici que l'on peut ajuster les seuils de la "carte de chaleur" (luminance minimale et maximale) pour analyser finement une zone de la pièce.

Ensuite, un deuxième onglet donnera place au catalogue de luminaires. L'interface permet de sélectionner une lampe IES et de la placer dans la scène d'un simple clic. J'ai programmé un système de repères visuels et de champs d'édition qui permettent de déplacer et changer la couleur ou l'intensité de la lampe à la volée. Un système de marqueurs visuels guide même l'utilisateur pour retrouver facilement les lampes placées si elles se trouvent hors du champ de vision de la caméra.

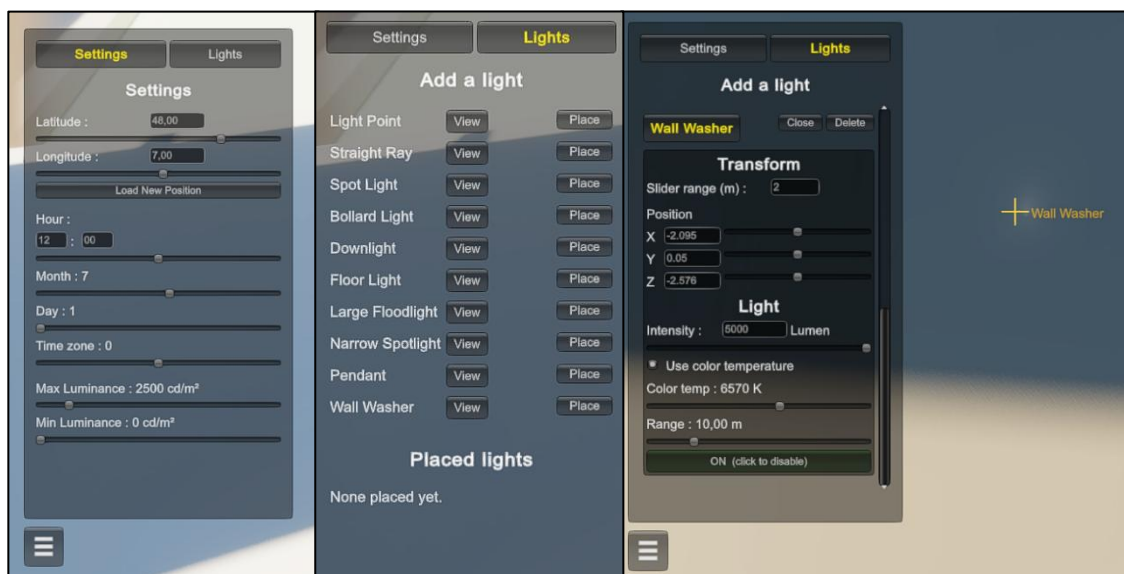


Figure 11 : Aperçu de l'interface utilisateur

Enfin, une interface sert de pont vers l'immersion en début de jeu : un simple bouton permet de basculer du mode "caméra simple" (pour un usage classique sur écran d'ordinateur) au mode "CAVE" (avec les 3 écrans).

4.2 - Le portage stéréoscopique pour le CAVE

L'infrastructure du laboratoire ICube comprend un CAVE (*Cave Automatic Virtual Environment*) : un dispositif de réalité virtuelle où les images sont projetées sur plusieurs murs, équipant l'utilisateur de lunettes 3D pour une immersion totale (voir figure 12). Ce dispositif nécessitait de diviser l'affichage sur 3 caméras distinctes, et de calculer deux images par écran (une pour l'œil gauche, une pour l'œil droit, pour créer l'effet de profondeur (stéréoscopie)).

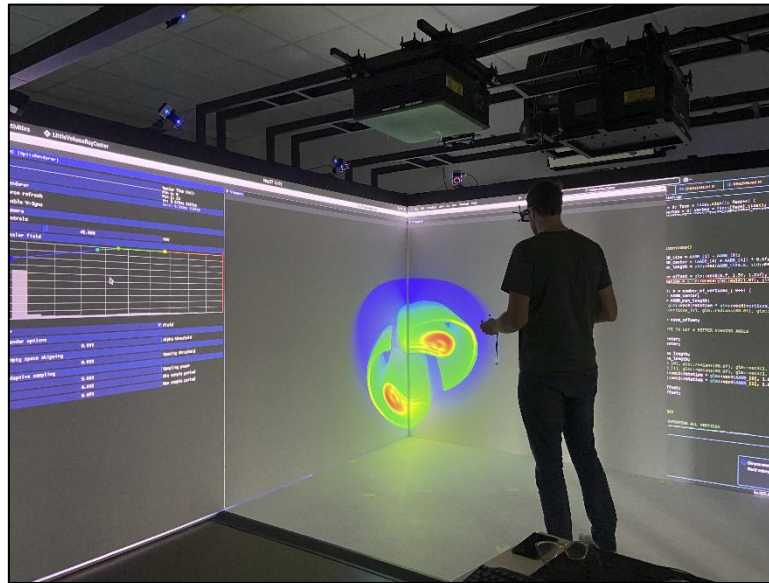


Figure 12 : Aperçu du CAVE et de ses trois écrans

Calculer du Path Tracing (déjà très gourmand sur un seul écran) 6 fois simultanément en temps réel était impossible. J'ai donc dû concevoir une architecture logicielle sur mesure pour contourner ce problème, basée sur un système de "gel" de l'image.

Le processus est le suivant :

- Lorsque l'utilisateur se déplace physiquement dans le CAVE, le moteur utilise une illumination globale allégée (RTGI). L'affichage reste fluide pour éviter une sensation de malaise.
- Lorsqu'il trouve un point de vue intéressant, il déclenche une capture. L'application se fige instantanément (le curseur est libéré, l'utilisateur peut analyser la scène). Techniquement, pour économiser drastiquement les ressources, les caméras en temps réel sont désactivées et remplacées par l'affichage d'images statiques via des Render Textures (des textures sur lesquelles l'image capturée a été projetée). Cela libère la carte graphique du rendu de l'environnement 3D, lui permettant de se concentrer exclusivement sur les lourds calculs du Path Tracing.
- En arrière-plan, mon script gérant l'affichage et la stéréoscopie prend le relais. Il va forcer le rendu de l'œil gauche, accumuler les rayons du Path Tracing pour la lumière naturelle, puis pour la lumière totale, avant de faire de même pour l'œil droit.

La carte de chaleur a également dû être adaptée. J'ai fait en sorte que la colorisation des pixels soit calculée indépendamment pour l'œil gauche et l'œil droit, afin que l'architecte puisse lire les valeurs de luminance tout en conservant la perception de profondeur de la maquette.

Grâce à ce système hybride, l'utilisateur bénéficie d'une navigation fluide à l'échelle humaine, tout en ayant accès à la qualité photoréaliste et scientifique du Path Tracing à la demande.

V - Résultats

Au terme de ces deux mois sur ce projet, le résultat est un prototype fonctionnel et autonome de visualisation architecturale. Ce prototype valide la faisabilité technique d'utiliser un moteur de jeu vidéo en temps réel pour des simulations lumineuses à visée scientifique et muséographique.

D'un point de vue visuel, le passage au pipeline HDRP couplé au Path Tracing a permis d'atteindre un niveau de photoréalisme particulièrement élevé. Les matériaux configurés lors de la phase de préparation (notamment le verre laissant diffuser la lumière) réagissent de manière physiquement correcte aux rebonds lumineux. L'outil génère une belle image et produit une véritable simulation physique (voir figure 13 et 14).

L'intégration réussie des données météorologiques de l'API PVGIS et du système de positionnement solaire permet d'obtenir un rendu naturel fidèle à la réalité géographique du bâtiment. La combinaison de cet éclairage naturel avec les profils photométriques IES des lumières artificielles offre un rendu hybride précis.

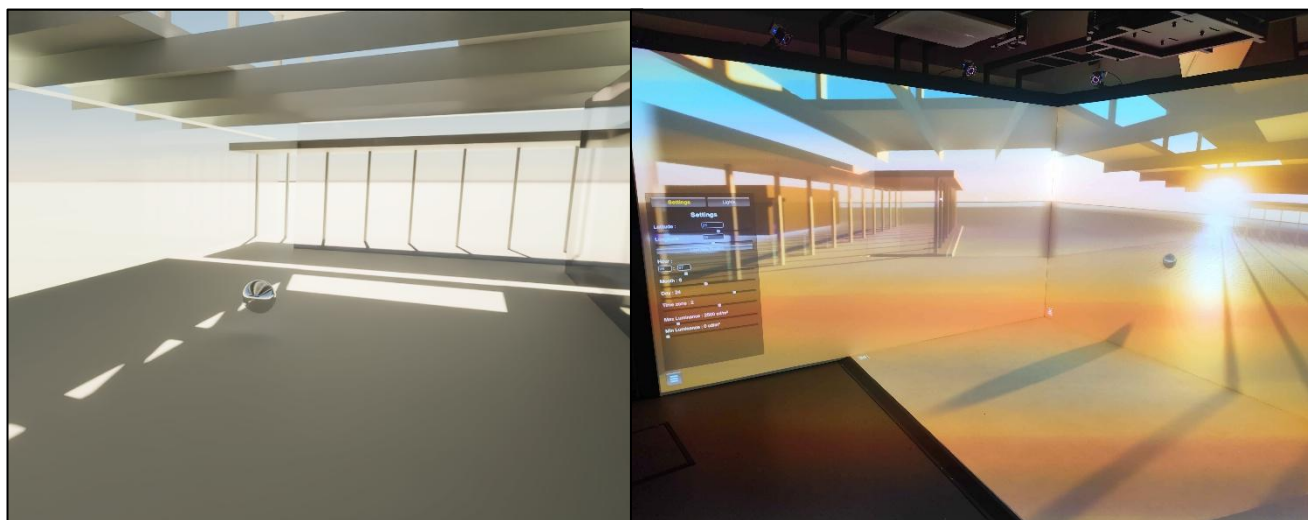


Figure 13 : Aperçu du rendu final dans la maquette muséale en mode simple caméra et CAVE



Figure 14 : Aperçu du rendu final de la carte de chaleur dans la maquette muséale en mode simple caméra et CAVE

Finalement, cet outil répond directement à la problématique initiale posée. Le prototype permet d'observer concrètement la répartition de la lumière dans l'espace muséal, offrant ainsi un support de décision crucial. Il facilite le dialogue entre les concepteurs et les conservateurs pour prévenir la dégradation des œuvres due à une surexposition, tout en maximisant l'usage de la lumière naturelle pour réduire l'empreinte carbone du bâtiment.

VI - Analyse et discussion

1 – Adéquation avec le besoin métier

Le prototype développé répond de manière concrète à la problématique initiale posée par le projet de recherche et l'architecte partenaire. Dans le domaine de l'architecture muséale, la lumière est une question d'esthétique mais aussi un enjeu de conservation des œuvres (qui se dégradent sous une exposition prolongée) et d'efficacité énergétique.

L'outil conçu s'intègre parfaitement dans ce besoin métier car il ne se contente pas de produire une image réaliste, il fournit des données quantifiables. Grâce à l'utilisation des profils photométriques standardisés (IES) et des données météorologiques réelles (API PVGIS), l'architecte dispose d'une simulation adéquate pour la conception de bâtiments. L'interface utilisateur, pensée pour des non-programmeurs, permet aux concepteurs d'interagir facilement avec la maquette pour tester différentes configurations lumineuses (ratio naturel/artificiel, positionnement des lampes) et ainsi prendre des décisions éclairées bien avant la construction ou la rénovation d'un bâtiment.

Bien que cet outil soit pour l'instant un prototype expérimental visant à valider les capacités analytiques du moteur Unity, il a suscité un vif intérêt lors de sa présentation. Il pose des bases techniques solides démontrant qu'une application dans le « vrai » monde de la conception architecturale est techniquement viable à court terme.

2 – Défis techniques et solutions apportées

La réalisation de ce prototype a nécessité de surmonter plusieurs obstacles techniques majeurs, nécessitant des compromis entre précision scientifique et performances informatiques.

L'un des premiers défis a été de s'assurer que le moteur de rendu, conçu initialement pour le jeu vidéo, pouvait servir d'outil analytique. L'utilisation du pipeline HDRP couplé au Path Tracing a permis d'obtenir une physique de la lumière réaliste. Pour prouver cette exactitude, j'ai mis en place une scène "boîte noire" et développé un compute shader générant une carte de chaleur. Cela m'a permis d'isoler et de valider mathématiquement la justesse des apports lumineux.

Il ne suffisait pas de placer une lumière directionnelle pour le soleil. J'ai dû implémenter des formules astronomiques complexes pour le positionnement solaire et utiliser un parseur de fichiers JSON en C# pour exploiter les données climatiques historiques de l'API européenne PVGIS, liant ainsi virtuellement la maquette à son emplacement géographique réel.

Si le développement sur écran classique a pleinement atteint ses objectifs, le portage stéréoscopique pour le CAVE a constitué le défi technique le plus complexe. Pour conserver des performances acceptables et éviter la surchauffe du GPU, j'ai imaginé un système de "photographie" à la demande. Techniquement, pour économiser les ressources lors d'une capture, les caméras en temps réel sont désactivées et remplacées par l'affichage d'images statiques via des Render Textures. Cela libère la carte graphique du rendu de l'environnement 3D pour la concentrer exclusivement sur les lourds calculs du Path Tracing.

Il est important de souligner que la compatibilité VR était un objectif secondaire du projet. Bien que le système hybride ait permis de contourner les limites matérielles, l'expérience dans le CAVE nécessite encore des optimisations. Actuellement, les performances lors des déplacements en stéréoscopie restent parfois insuffisantes (framerate instable et insuffisant). De plus, le rendu figé présente encore quelques imperfections au niveau de l'alignement de la stéréoscopie entre l'œil gauche et l'œil droit et les contrôles pour se déplacer dans la maquette n'ont pas encore été appliqués sur les manettes du CAVE (actuellement, le déplacement et l'interaction en jeu se fait uniquement en clavier / souris). Le temps imparti de deux mois ne m'a pas permis de pousser plus loin ces optimisations, ce qui nécessiterait une refonte plus profonde de la gestion multi-caméras dans Unity, offrant ainsi une bonne piste d'amélioration future.

3 – Bilan personnel et compétences acquises

Ce stage de deux mois au sein de l'équipe IGG du laboratoire ICube a été extrêmement formateur, tant sur le plan technique que professionnel, me permettant de faire la synthèse entre mes enseignements universitaires et les exigences du monde de la recherche.

D'un point de vue technique, ce projet a été l'occasion de mettre en pratique et d'enrichir directement les connaissances acquises lors de ma licence. L'utilisation d'Unity s'est imposée naturellement dans la continuité du cours de *Moteurs de jeux 3D* (dispensé par mon tuteur, Paul VIVILLE), tandis que la préparation des maquettes sur Blender a mobilisé mes acquis des modules *Infographie 3D* et *Animation 3D*. Ce projet a également été un excellent terrain d'application pour les concepts vus en *Génie Logiciel*, m'amenant à structurer rigoureusement mon travail via l'utilisation d'un dépôt Git. Au-delà de ces fondations, j'ai perfectionné ma pratique du C# en autodidacte et me suis confronté à des concepts avancés de l'informatique graphique qui m'étaient inédits : l'exploitation du pipeline HDRP, le Path Tracing, la programmation de compute shaders exécutés sur le GPU, ou encore la communication avec des API web externes.

Sur le plan humain et organisationnel, travailler à nouveau dans un laboratoire de recherche a renforcé ma rigueur scientifique (notamment dans la méthodologie de test et la validation des données). J'ai gagné en autonomie face à des problèmes techniques non documentés, tout en apprenant à communiquer efficacement avec mon tuteur et mes collègues stagiaires pour leurs avis et conseils. Enfin, concevoir une interface en pensant aux besoins spécifiques d'un utilisateur final m'a sensibilisé à l'importance de l'ergonomie et de l'expérience utilisateur dans le développement de logiciels.

Enfin, ce stage a eu un impact décisif sur mon projet professionnel. Il me conforte pleinement dans mon choix de poursuivre en Master Image & 3D et d'orienter ma carrière vers ce domaine d'expertise. Par ailleurs, cette immersion réussie au sein du laboratoire ICube m'a réellement fait découvrir le monde de la recherche académique sous un angle très stimulant. Bien que ce ne soit pas mon objectif premier, envisager une poursuite vers la recherche dans le domaine de l'image de synthèse est aujourd'hui une perspective que je n'exclus pas et qui me plairait beaucoup.

VII - Conclusion

Ce stage de deux mois au sein de l'équipe IGG du laboratoire ICube a été l'occasion de développer, de sa conception à son implémentation, un prototype de visualiseur de lumières naturelles et artificielles destiné à l'architecture. Face aux enjeux écologiques actuels et aux problématiques spécifiques de la conservation muséale, l'objectif était de fournir aux architectes un outil d'aide à la décision fiable, immersif et scientifiquement rigoureux.

Grâce à l'exploitation des technologies récentes du moteur Unity, notamment le pipeline HDRP et le Path Tracing, j'ai pu démontrer qu'il est possible d'utiliser un moteur de jeu vidéo en temps réel à des fins de simulation physique précise. L'intégration de données météorologiques réelles, l'utilisation de profils photométriques standardisés et le développement d'outils d'analyse quantitative, comme la carte de chaleur par compute shader, font de ce prototype une base solide répondant directement au cahier des charges initial.

Si les résultats sur écran standard sont très concluants, ce projet ouvre également la voie à de nombreuses perspectives d'amélioration. À court terme, la refonte du système de rendu stéréoscopique permettrait d'offrir une expérience de réalité virtuelle totalement fluide dans le CAVE, sans avoir recours au système de capture figée. À plus long terme, le prototype pourrait être enrichi par le développement de modules d'importation automatisés pour des maquettes architecturales complexes (comme le format IFC), ou encore par l'intégration de technologies basées sur l'intelligence artificielle (comme le DLSS) pour le débruitage et l'optimisation des performances en temps réel.

D'un point de vue personnel, cette expérience au contact du monde de la recherche m'a permis de consolider mes acquis académiques et de développer de nouvelles compétences techniques en informatique graphique. Ce stage constitue une étape déterminante dans mon cursus CMI IIRVIJ, confirmant mon attrait pour la réalité virtuelle et la programmation graphique orientée vers des applications métiers concrètes.

VIII – Références bibliographiques

- [1] ICube - ICube. (s. d.). <https://icube.unistra.fr/>
- [2] Équipe IGG : Informatique géométrique et graphique. (s. d.). <https://igg.icube.unistra.fr/index.php/Accueil>
- [3] Technologies, U. (s. d.). Unity - Manual : Unity 6.0 User Manual. <https://docs.unity3d.com/6000.0/Documentation/Manual/UnityManual.html>
- [4] High Definition Render Pipeline overview | High Definition RP | 8.0.1. (s. d.). <https://docs.unity3d.com/Packages/com.unity.render-pipelines.high-definition@8.0/manual/HDRP-Features.html>
- [5] Nico-Lmy. (s. d.). GitHub - nico-lmy/Photorealistic-Renderings-of-Architectural-Models : Développement dans Unity d'un prototype de visualiseur de lumières naturelles et artificielles pour l'architecture. GitHub. <https://github.com/nico-lmy/Photorealistic-Renderings-of-Architectural-Models>
- [6] Contributeurs aux projets Wikimedia. (2026a, avril 29). Position du soleil dans le ciel. https://fr.wikipedia.org/wiki/Position_du_soleil_dans_le_ciel
- [7] Sustainable By Design : Equation of Time. (s. d.). <https://www.susdesign.com/popups/sunangle/eot.php>
- [8] API non-interactive service. (s. d.). Joint Research Centre. https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis/using-pvgis-5/api-non-interactive-service_en
- [9] PVGIS typical meteorological year (TMY) generator. (s. d.). Joint Research Centre. https://joint-research-centre.ec.europa.eu/photovoltaic-geographical-information-system-pvgis/using-pvgis-5/pvgis-5-tools/pvgis-typical-meteorological-year-tmy-generator_en
- [10] Contributeurs aux projets Wikimedia. (2026, 16 juin). Luminance. <https://fr.wikipedia.org/wiki/Luminance>
- [11] IES-Library : Browse. (s. d.). <https://ieslibrary.com/browse>